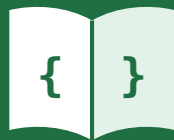

Module 23 — Spring Cloud

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 The System of This Module	2
2.2 Spring Cloud or Kubernetes-Native? A Decision Guide	3
3. Step-by-Step Examples	4
3.1 The Gateway	4
3.2 HTTP Interface or OpenFeign?	5
3.3 Spring Cloud LoadBalancer	6
3.4 Circuit Breaker and Fallback	7
3.5 Config Server	9
3.6 The Whole System with Compose	10
3.7 Spring Cloud Kubernetes	11
4. Common Mistakes and Best Practices	13
5. Summary	13
6. Further Reading	13

1. Learning Goals

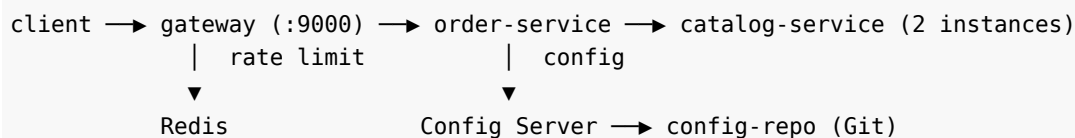
By the end of this module you will be able to:

- Put an API gateway in front of services: routes, filters and rate limiting with Spring Cloud Gateway
- Serve configuration for all services from a Config Server with a Git backend, and refresh it without a restart
- Call another service with an HTTP interface or with OpenFeign, and compare the two
- Balance calls over several instances with Spring Cloud LoadBalancer
- Protect callers with a circuit breaker (Resilience4j) and a fallback
- Use Kubernetes itself for configuration and discovery with Spring Cloud Kubernetes
- Decide which of these problems Spring Cloud should solve, and which the platform

Prerequisites: Module 04 (HTTP clients, resilience), Module 15 (Actuator), Module 22 (Kubernetes) · **Estimated time:** 6 hours · **Docker required;** kind for section 3.7

2. Concepts

2.1 The System of This Module



Folder	Service	Topics
services/gateway	API gateway	routes, filters, rate limiting (Redis)
services/config-server	Config Server	Git backend, profiles
lesson	order service	HTTP interface vs Feign, LoadBalancer, circuit breaker, refresh, Spring Cloud Kubernetes
services/catalog-service	catalog	the service that is called; can simulate failures

2.2 Spring Cloud or Kubernetes-Native? A Decision Guide

Spring Cloud solves the typical problems of distributed systems inside the application. Kubernetes (and service meshes like Istio or Linkerd) solve many of the same problems in the platform. Neither is “right”. What matters is that each problem is solved once, in one place.

Problem	Spring Cloud (in the application)	Kubernetes / platform	Choose the platform when ...
Service discovery	DiscoveryClient + LoadBalancer	Service + DNS	you run on Kubernetes: a Service name is enough
Load balancing	client-side, per request (LoadBalancer)	kube-proxy per connection; per request with a mesh	you do not need client-side rules (zones, hints)
Configuration	Config Server (Git, history, encryption)	ConfigMap / Secret	configuration is per environment and deployed with the manifests
Configuration refresh	/actuator/refresh, @RefreshScope	rolling restart (generators, module 22) or Spring Cloud Kubernetes reload	a restart is cheap and safe
Edge / API gateway	Spring Cloud Gateway (Java filters, custom logic)	Ingress / Gateway API controller	you only need routing, TLS and simple limits
Resilience	Resilience4j (fallback in code)	mesh retries and timeouts	the reaction does not need business logic (a fallback does)

Rules of thumb:

- **Outside Kubernetes** (VMs, Docker Compose), Spring Cloud gives you what the platform does not.
- **On Kubernetes**, use the platform for discovery and basic configuration. Keep Spring Cloud where the application must decide something: a fallback with business meaning, a gateway filter with application logic, configuration history across many services.
- **Spring Cloud Kubernetes** is the bridge: the same Spring abstractions (DiscoveryClient, property sources, refresh), with Kubernetes as the backend.

3. Step-by-Step Examples

Start the whole system with Docker Compose. The script builds the four services first:

```
modules/23-spring-cloud/up.sh
```

```
bookstore-cloud-gateway-1      Up (healthy)    0.0.0.0:9000->8080/tcp
bookstore-cloud-order-service-1 Up (healthy)    8080/tcp
bookstore-cloud-catalog-1-1    Up (healthy)    8080/tcp
bookstore-cloud-catalog-2-1    Up (healthy)    8080/tcp
bookstore-cloud-config-server-1 Up (healthy)    8080/tcp
bookstore-cloud-redis-1        Up (healthy)    6379/tcp
```

Only the gateway publishes a port. [requests.http](#) contains all requests of this section.

3.1 The Gateway

The routes of `services/gateway/src/main/resources/application.yaml` (under `spring:`):

```
cloud:
  gateway:
    server:
      webflux:
        default-filters:
          - AddResponseHeader=X-Served-Through, bookstore-gateway
        routes:
          - id: catalog
            uri: lb://catalog-service           # lb:// → Spring Cloud LoadBalancer
            ↪ picks an instance
            predicates:
              - Path=/api/books/**
            filters:
              - name: RequestRateLimiter       # token bucket in Redis, shared by all
            ↪ gateway instances
              args:
                redis-rate-limiter.replenishRate: 5    # tokens per second
                redis-rate-limiter.burstCapacity: 10   # at most 10 requests at once
                key-resolver: "#{@clientIpKeyResolver}"
          - id: orders
            uri: lb://order-service
            predicates:
              - Path=/api/orders/**
              - Method=POST
```

- A **route** has predicates (here the path) and a target. `lb://catalog-service` asks the load balancer for an instance.

- **Filters** change the request or the response. default-filters apply to all routes.
- RequestRateLimiter is a token bucket in Redis. Several gateway instances share it.

The key decides whose requests are counted together:

```
@Configuration(proxyBeanMethods = false)
class RateLimitConfiguration {

    @Bean
    KeyResolver clientIpKeyResolver() {
        return exchange ->
            ↪ Mono.just(Optional.ofNullable(exchange.getRequest().getHeaders().getFirst("X-
            ↪ Customer")))
                .or(() -> Op-
                ↪ tional.ofNullable(exchange.getRequest().getRemoteAddress()).map(InetSocketAddress:
                .orElse("unknown")));
    }
}
```

```
X-RateLimit-Remaining: 5
X-RateLimit-Burst-Capacity: 10
X-RateLimit-Replenish-Rate: 5
X-Served-Through: bookstore-gateway
```

When the bucket is empty, the gateway answers 429 Too Many Requests and does not call the service:

```
@Test
void tooManyRequestsAreRejected() {
    List<Integer> statuses = new ArrayList<>();
    for (int i = 0; i < 20; i++) { // burst capacity 10, 5 new tokens
        ↪ per second
        statuses.add(http.get().uri("/api/books/9780134685991").header("X-Customer",
        ↪ "rate-limit-test").exchange()
            .returnResult(String.class).getStatus().value());
    }

    assertThat(statuses).contains(200, 429); // 429 Too Many Requests
}
```

3.2 HTTP Interface or OpenFeign?

The order service asks the catalog for the price. The same client, written two ways:

```
@HttpExchange("/api/books")
public interface CatalogHttpClient {

    @GetExchange("/{isbn}")
    Book find(@PathVariable String isbn);
}
```

```
@FeignClient(name = "catalog-service", path = "/api/books")    // name = the service ID,
↳ resolved by the load balancer
public interface CatalogFeignClient {

    @GetMapping("/{isbn}")
    Book find(@PathVariable String isbn);
}
```

	HTTP interface	OpenFeign
Part of	Spring Framework (6+)	Spring Cloud (Netflix heritage)
Annotations	@HttpExchange, @GetExchange	Spring MVC annotations (@GetMapping)
Underlying client	RestClient / WebClient (sync or reactive)	Feign's own client, blocking
Load balancing	@LoadBalanced builder (section 3.3)	built in (name = service ID)
Status	the recommended way for new code	"feature-complete" since 2022.0: only bug fixes; the Spring team suggests HTTP service clients

Both give the same book (LoadBalancedClientsTest). Use HTTP interfaces for new code. Feign is still common in existing systems.

3.3 Spring Cloud LoadBalancer

The HTTP interface gets its load balancing from a @LoadBalanced RestClient.Builder. The host of http://catalog-service is a service ID, not a DNS name:

```
@Configuration(proxyBeanMethods = false)
class CatalogClientConfiguration {
```

```

@Bean
@LoadBalanced // the host name is a service ID,
↳ not a DNS name
RestClient.Builder loadBalancedRestClientBuilder() {
    return RestClient.builder();
}

@Bean
CatalogHttpClient catalogHttpClient(RestClient.Builder
↳ loadBalancedRestClientBuilder) {
    RestClient restClient =
↳ loadBalancedRestClientBuilder.baseUrl("http://catalog-service").build();
    return HttpServiceProxyFac-
↳ tory.builderFor(RestClientAdapter.create(restClient)).build()
        .createClient(CatalogHttpClient.class);
}
}

```

Outside Kubernetes, the instances come from a fixed list (SimpleDiscoveryClient). On Kubernetes, the cluster provides them (section 3.7):

```

# Lesson 3.3 – outside Kubernetes: a fixed list of instances (spring-boot:run,
↳ compose.yaml)
spring:
  config:
    activate:
      on-profile: "!kubernetes"
  cloud:
    discovery:
      client:
        simple:
          instances:
            catalog-service:
              - uri: ${CATALOG_1_URL:http://localhost:8081}
              - uri: ${CATALOG_2_URL:http://localhost:8082}

```

Call the catalog four times through the gateway: servedBy alternates between catalog-1 and catalog-2 (round robin).

3.4 Circuit Breaker and Fallback

When the catalog fails, the order service should neither wait nor fail. It accepts the order as PENDING and prices it later:

```

public OrderResponse place(OrderRequest request, String client) {
    if (request.quantity() > limits.maxQuantity()) {
        throw new IllegalArgumentException("At most " + limits.maxQuantity() + " copies
        ↪ per order");
    }
    Supplier<Book> call = "feign".equals(client)
        ? () -> feignClient.find(request.isbn())
        : () -> httpClient.find(request.isbn());
    return catalogBreaker.run(
        () -> priced(request, call.get(), client),
        error -> pending(request, client));    // fallback: failure, timeout, or
    ↪ the breaker is open
}

```

```

resilience4j:
  circuitbreaker:
    instances:
      catalog:
        sliding-window-size: 4           # look at the last 4 calls ...
        minimum-number-of-calls: 4
        failure-rate-threshold: 50      # ... and open when half of them failed
        wait-duration-in-open-state: 10s # then try again after 10 s (HALF_OPEN)
    timelimiter:
      instances:
        catalog:
          timeout-duration: 1s          # a slower answer counts as a failure

```

State	Behaviour
CLOSED	calls go through; failures are counted
OPEN	calls fail at once, the fallback answers, the catalog is not called
HALF_OPEN	after wait-duration-in-open-state, a few test calls decide: back to CLOSED, or OPEN again

```

@Test
void failuresOpenTheBreakerAndTheFallbackAnswers() {
    CatalogInstances.failing();

    for (int i = 0; i < 4; i++) {
        // minimum-number-of-calls: 4
        assertThat(orders.place(new OrderRequest("9780134685991", 1), "http").status())
            .isEqualTo(OrderResponse.Status.PENDING);
    }
}

```

```

    }

    assert-
    ↪ That(breakers.circuitBreaker("catalog").getState()).isEqualTo(CircuitBreaker.State.OPEN);
    int callsBefore = CatalogInstances.CATALOG_1.getAllServeEvents().size()
        + CatalogInstances.CATALOG_2.getAllServeEvents().size();
    orders.place(new OrderRequest("9780134685991", 1), "http");
    int callsAfter = CatalogInstances.CATALOG_1.getAllServeEvents().size()
        + CatalogInstances.CATALOG_2.getAllServeEvents().size();
    assertThat(callsAfter).isEqualTo(callsBefore);           // open: the catalog is not
    ↪ called at all
}

```

Try it with the running system: switch both catalog instances to failing (see requests.http), send orders, and look at /actuator/circuitbreakers of the order service: "state":"OPEN".

3.5 Config Server

```

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

```

```

cloud:
  config:
    server:
      git:
        # the course repository itself (run from this folder); in production: the URL of
    ↪ a config repository
        uri: ${CONFIG_GIT_URI:file://${user.dir}/../../../../../}
        search-paths: modules/23-spring-cloud/config-repo
        default-label: main           # the branch – only committed changes are visible
        clone-on-start: true

```

The server answers GET /order-service/default with the content of order-service.yaml (and application.yaml for all services) from config-repo/. A profile adds order-service-prod.yaml. ConfigServerTest creates a temporary Git repository with JGit and checks both.

The client imports the configuration at startup:

```
config:
  import: optional:configserver:${CONFIG_SERVER_URL:http://localhost:8888} # optional:
  ↪ starts without it
```

A value is read again after a refresh only by beans in the **refresh scope**:

```
@Component
@RefreshScope
public class OrderLimits {

    private final int maxQuantity;

    OrderLimits(@Value("${bookstore.order.max-quantity:10}") int maxQuantity) {
        this.maxQuantity = maxQuantity;
    }

    public int maxQuantity() {
        return maxQuantity;
    }
}
```

```
# change bookstore.order.max-quantity in config-repo/order-service.yaml (and commit,
↪ with the Git backend)
docker exec bookstore-cloud-order-service-1 wget -qO- --post-data='
↪ http://localhost:8080/actuator/refresh
["bookstore.order.max-quantity"]
```

i Note

With the Git backend, the server sees only **committed** changes: configuration gets a history, reviews and rollbacks, like code. The compose containers use the native profile (a mounted folder) instead, so that the demo needs no Git inside the container.

! Warning

On macOS, /var is a symbolic link, and the Config Server refuses repository paths with symbolic links. The test therefore uses `toRealPath()` for its temporary repository.

3.6 The Whole System with Compose

`compose.yaml` builds all services with one Dockerfile (the one from module 20), and starts them in the right order with health checks: Redis and the Config Server first, the gateway last. The services find each other by their compose names (`http://catalog-1:8080`).

Stop the system with `docker compose -f modules/23-spring-cloud/compose.yaml down`.

3.7 Spring Cloud Kubernetes

On Kubernetes, the order service needs neither the Config Server nor the fixed instance list. Spring Cloud Kubernetes reads both from the Kubernetes API:

```
# Lesson 3.7 – on Kubernetes: the ConfigMap "order-service" is a property source, the
↪ pods of the Service
# "catalog-service" are the instances, and a changed ConfigMap refreshes the application
↪ without a restart
spring:
  config:
    activate:
      on-profile: kubernetes
    import: "kubernetes:"
  cloud:
    config:
      enabled: false                # no Config Server on Kubernetes
    kubernetes:
      reload:
        enabled: true
        mode: event                # watch the ConfigMap (needs the "watch" permission)
      loadbalancer:
        mode: POD                  # the pods behind the Service are the instances:
↪ Spring Cloud balances
```

The ConfigMap has the name of the application:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-service              # = spring.application.name: read as a
↪ property source
data:
  application.yaml: |
    bookstore:
      order:
        max-quantity: 4
```

The pod reads the Kubernetes API, so it needs permissions:

```
apiVersion: v1
kind: ServiceAccount
metadata:
```

```

    name: order-service
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: spring-cloud-kubernetes
rules:
  - apiGroups: [""]
    resources: ["configmaps", "services", "endpoints", "pods"]
    verbs: ["get", "list", "watch"]           # watch: for the reload of changed
    ↪ ConfigMaps
  - apiGroups: ["discovery.k8s.io"]
    resources: ["endpointslices"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: order-service-reads-the-api
subjects:
  - kind: ServiceAccount
    name: order-service
roleRef:
  kind: Role
  name: spring-cloud-kubernetes
  apiGroup: rbac.authorization.k8s.io

```

```

scripts/kind-up.sh
modules/23-spring-cloud/up.sh           # builds the images (the compose system can be
    ↪ stopped afterwards)
modules/23-spring-cloud/deploy-k8s.sh
kubectl -n bookstore-cloud port-forward service/order-service 18090:8080

```

What we observed on kind:

- An order for 5 copies: At most 4 copies per order (the value from the ConfigMap).
- Four orders: pricedBy alternates between the two catalog pods (mode: POD).
- `kubectl -n bookstore-cloud patch configmap order-service ...` with `max-quantity: 6`: a few seconds later, 5 copies are accepted, without a restart (reload.mode: event).

★ Tip

With `loadbalancer.mode: SERVICE`, the order service calls the Kubernetes Service, and kube-proxy balances **per connection**. Because HTTP connections are kept alive, all calls went to the same pod. POD mode lets Spring Cloud LoadBalancer choose per request.

4. Common Mistakes and Best Practices

!! Caution

A fallback must be correct for the business, not just “something”. A PENDING order that is priced later is honest. A made-up price is not.

- **Do:** solve each problem in one place: discovery by Kubernetes or by Spring Cloud, not both at the same time for the same calls.
- **Don't:** put business logic into the gateway. It routes, authenticates and limits. Orders are the order service's job.
- **Do:** give every remote call a timeout (here the `TimeLimiter`, 1 s). A circuit breaker without a timeout does not protect against slow services.
- **Don't:** expose `/actuator/refresh` publicly. It changes the running system.
- **Do:** keep secrets out of the Config Server's plain files (use its encryption, Vault, or Kubernetes Secrets).
- **Don't:** start new code with Feign. HTTP interfaces are the Spring Framework standard.

5. Summary

- Spring Cloud Gateway routes by predicates, changes requests with filters, and limits rates with a token bucket in Redis.
- The Config Server serves configuration from Git. `@RefreshScope` beans get new values after `/actuator/refresh`.
- HTTP interfaces and OpenFeign describe clients as interfaces. `@LoadBalanced` and `lb://` resolve service IDs to instances.
- A circuit breaker counts failures, opens, and lets a fallback answer until the service recovers.
- Spring Cloud Kubernetes uses ConfigMaps and the Kubernetes API with the same Spring abstractions.
- On Kubernetes, prefer the platform for discovery and simple configuration, and keep Spring Cloud where the application decides.

Next step: [Exercises](#)

6. Further Reading

- [Spring Cloud Gateway](#) · [Spring Cloud Config](#)
- [Spring Cloud Commons — LoadBalancer](#)
- [Spring Cloud Circuit Breaker](#) · [Spring Cloud OpenFeign](#)
- [Spring Framework — HTTP Interface Clients](#)

- [Spring Cloud Kubernetes](#)